

Spring Semester 2026 EE209 Midterm Exam

Pledging of No Cheating

Note: Please write down your student ID and name, sign on it (draw your signature), and date it. If you do not fill out this page, we won't grade your mid-term exam.

저는 이번 시험을 치르면서 이 과목에서 금지한 어떤 부정행위도 저지르지 않을 것임을 서약합니다. 추후에 위반사항이 발견되었을 경우 합당한 모든 불이익을 감수하겠습니다.

I pledge that I will not participate in any activity of cheating disallowed by this course while taking this exam online. I will assume full responsibility if any violation is found later.

Student ID: _____ Name: _____

Signature: _____ Date: _____

The exam is closed book and notes. Read the questions carefully and focus your answers on what has been asked. You are allowed to ask the instructor/TAs for help only in understanding the questions, in case you find them not completely clear. Be concise and precise in your answers and state clearly any assumption you may have made. All your answers must be included in the attached sheets. You have 150 minutes to complete your exam. Be wise in managing your time.

Please do not fill in the "Score" fields below. Self-grading is not allowed. Good luck!

1	/10
2	/10
3	/10
4	/10
5	/10
6	/10
7	/10
8	/10
9	/10
Total	/90

1. C Fundamentals

① (1 point for each) True or False

- i. Overflow in unsigned integers is well-defined in C language.
Your answer: (True / False)
- ii. In two's complement, there are two representations for zero.
Your answer: (True / False)
- iii. In an expression mixing signed and unsigned integers, the signed integer is converted to unsigned.
Your answer: (True / False)

iv. sizeof(int) returns a signed integer.

Your answer: (True / False)

② (2 points) What is the output of the following code?

```
#include <stdio.h>

int main() {
    int x = 2;
    switch(x) {
        case 1: printf("A ");
        case 2: printf("B ");
        case 3: printf("C ");
        default: printf("D");
    }

    return 0;
}
```

Your answer: _____

③ (2 points) What is the output of the following code?

```
#include <stdio.h>

int main() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (j == 1) break;
            printf("%d%d ", i, j);
        }
    }

    return 0;
}
```

Your answer: _____

- ④ (2 points) What is the output of the following code?

```
#include <stdio.h>

int main() {
    int a[] = {1, 2, 3, 4};
    int *p = a;

    printf("%d\n", *(p + *(p + 2)));
    return 0;
}
```

Your answer: _____

2. Numbers in Computers

- ① (1 point) Convert decimal 13 to binary.

Your answer: _____

- ② (1 point) Convert the binary 10110_2 to decimal.

Your answer: _____

- ③ (1 point) Convert the binary 11101010_2 to hexadecimal.

Your answer: _____

- ④ (1 point) Find the 8-bit 2's complement representation of -5 in binary.

Your answer: _____

- ⑤ (1 point) Compute the result of $-50 + (-80)$ in decimal, assuming both numbers are represented as 8-bit signed integers (2's complement).

Your answer: _____

- ⑥ (1 point) What is the result of `printf("%d\n", -10 < 5U);`?

Your answer: _____

- ⑦ (1 point) Does the following code detect overflow correctly?

```
int a = INT_MAX;
int b = a + 1;
if (b < a) printf("overflow\n");
```

Your answer: (Yes / No)

- ⑧ (1 point) What is the value of a?

```
int a = 0;
int x = 0;
if (x & 1 == 0)
    a = 1;
```

Your answer: _____

- ⑨ (1 point) What is the output of the following code?

```
unsigned int x = (unsigned int)-1;
int y = (int)x;
printf("%d\n", y);
```

Your answer: _____

- ⑩ (1 point) What is the output of the following code?

```
int a = 5, b = 3;  
a = a ^ b; b = a ^ b; a = a ^ b;
```

Your answer: a = _____ b = _____

3. C Data Types

- ① (1 point) Find a bug in the following code and fix it.

```
char c = getchar();  
if (c == EOF)  
    printf("EOF\n");
```

Your answer:

- ② (1 point) What is the output of the following code?

```
unsigned short u = 65535;  
printf("%hu\n", u); printf("%hd\n", u);
```

Your answer: _____

- ③ (1 point) What is the output of the following code?

```
char s[] = "abcde";  
printf("%lu %lu\n", sizeof(s), strlen(s));
```

Your answer: _____

- ④ (1 point for each) Determine the data type of each constant.

i. 123.456

Your answer: _____

ii. 123.456F
Your answer: _____

iii. 123.456L
Your answer: _____

iv. 1E-2
Your answer: _____

⑤ (1 point) Which statement is True?

- A. char is not an integer type
- B. Strings are a built-in type in C
- C. char can be used in arithmetic
- D. sizeof("a") == 1

Your answer: _____

⑥ (2 point) Explain the difference between 'a' vs "a".

Your answer:

4. Functions

① (2 points) What is the output of f(3)?

```
void f(int n) {  
    if (n == 0) return;  
    printf("%d ", n);  
    f(n - 1);  
    printf("%d ", n);  
}
```

Your answer: _____

- ② (2 points) Fill in the `blank` with recursion using only `f()` such that $f(5) = 8$, $f(10) = 89$, and $f(16) = 1,597$.

```
int f(int n) {  
    if (n <= 1) return 1;  
    return blank;  
}
```

Your answer: _____

- ③ (4 points) Fill in the `blank[a]` and `blank[b]` such that the following program prints 10.

```
int f(int *p, int n) {  
    if (n == 0) return 0;  
    return *p + f(p + 1, blank[a]);  
}  
  
int main() {  
    int a[] = {1, 2, 3, blank[b]};  
    printf("%d\n", f(a, 4));  
}  
  
output: 10
```

Your answer: `blank[a]`: _____ `blank[b]`: _____

- ④ (2 points) What is the output of $f(12)$?


```
int f(int n) {
    static int x = 0;
    if (n == 0) return x;
    x += n;
    return f(n - 1);
}
```

Your answer: _____

5. Scope, Array, and String

Consider the following two C files.

```
/* counter.c */
static int count = 0; /* line 1 */
void increment() {
    count++;
}
int get() {
    return count;
}
```

```
/* main.c */
#include <stdio.h>
extern int count;
void increment();
int get();
int main() {
    increment();
    increment();
    printf("%d %d\n", get(), count);
    return 0;
}
```

- ① (1 point) Will this program compile and link?

Your answer: _____

- ② (1 point) What will it print??

Your answer: _____

- ③ (1 point) What is the output of the program if static were removed from line 1 of counter.c?

Your answer: _____

- ④ (2 points) Briefly explain lexical scope and temporal scope in one sentence for each.

Your answer:

- ⑤ (2 points) What is the output of the following code?

```
int x;

void f() {
    int x = x + 1;
    printf("%d\n", x);
}

int main() {
    f();
    printf("%d\n", x);
    return 0;
}
```

Your answer: _____

- ⑥ (3 points) Identify which declaration of i is used at each marked line.

```
int i = 10;          /* Declaration A */
void g(void) {
    int i = 20;      /* Declaration B */
    if (i > 0) {
        int i;       /* Declaration C */
        i = 30;      /* Line X */
    }
    i = 40;          /* Line Y */
}
void h(void) {
    i = 50;          /* Line Z */
}
```

Your answer:

Line X:_____ Line Y:_____ Line Z:_____

6. Pointer

- ① (1 point) What is the type of the variable p in 'char (*p)[4];'?

Your answer: _____

- ② (1 point) Briefly describe what is wrong with the following code.

```
int a[3], b[3];
if (&a[1] > &b[1])
    printf("yes");
```

Your answer:

- ③ (2 points) What is the output of the following code?

```
int x = 10, y = 20;
int *p = &x;
int **q = &p;

*q = &y;
**q = 30;

printf("%d %d\n", x, y);
```

Your answer: _____

- ④ (2 points) Fill in the **blank** so that the function reverse() correctly reverses an array of integers in-place.

```
void reverse(int *a, int n) {
    int *p = a;
    int *q = blank;

    while (p < q) {
        int tmp = *p;
        *p = *q;
        *q = tmp;
        p++;
        q--;
    }
}
```

Your answer: _____

- ⑤ (2 points) Complete the two blanks `blank_[a]` and `blank_[b]` using pointer `p` so that the function `count_even()` correctly counts even numbers in the array.

```
int count_even(int *a, int n) {  
    int count = 0;  
    int *p = a;  
  
    for (int i = 0; i < n; i++) {  
        if (blank_[a] % 2 == 0)  
            count++;  
        blank_[b];  
    }  
    return count;  
}
```

Your answer: `blank_[a]`: _____ `blank_[b]`: _____

- ⑥ (2 point) What is the output of the following code?

<pre>#include <stdio.h> void update(int *p, int *q) { *p += 2; q++; *q += *p; } void shift(int **pp) { (*pp)++; **pp += 3; }</pre>	<pre>int main() { int A[4] = {1, 2, 3, 4}; int B[4] = {10, 20, 30, 40}; int *p = A; int *q = B; update(p, q); shift(&p); *(p + 1) += *(q + 2); *q += *p; printf("%d %d\n", A[0] + A[1] + A[2] + A[3], B[0] + B[1] + B[2] + B[3]); return 0; }</pre>
--	--

Your answer: _____

7. Structures

- ① (1 points) Fill in the blank to dynamically allocate exactly enough memory for one Student struct.

<pre>typedef struct { int id; char *name; } Student; Student *s = blank</pre>
--

Your answer: _____

- ② (1 point) What is the output of the following code?

```
typedef struct {  
    char *name;  
} S;  
  
int main() {  
    S s1, s2;  
    s1.name = "DEF";  
    s2 = s1;  
    s2.name = "321";  
    printf("%s%s\n", s1.name, s2.name);  
}
```

Your answer: _____

- ③ (2 points) Does the following code compile without errors? If not, identify the error and briefly explain its cause.

```
struct A { int x; };  
struct B { int x; };  
struct A a = {5};  
struct B b;  
b = a;
```

Your answer:

- ④ (2 points) Identify the bug in the following code and provide a concise explanation of the fix.

```
char **arr = malloc(3 * sizeof(char *));  
for (int i = 0; i < 3; i++)  
    arr[i] = malloc(10);  
  
free(arr);  
for (int i = 0; i < 3; i++)  
    free(arr[i]);
```

Your answer:

- ⑤ (4 point) What is the output of the following code?

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *name;
    int *scores;
} Student;

int main() {
    Student s1, s2;

    s1.name = malloc(10);
    strcpy(s1.name, "John");

    s1.scores = malloc(3 * sizeof(int));
    for (int i = 0; i < 3; i++)
        s1.scores[i] = i + 1;

    s2 = s1;
    s2.name = malloc(10);
    strcpy(s2.name, "Mike");
    s2.scores[0] = 100;

    printf("%s %d %s %d\n", s1.name, s1.scores[0], s2.name, s2.scores[0]);

    free(s1.name);
    free(s2.name);
    free(s1.scores);
}
```

Your answer: _____

8. Stacks and Queues

- ① (1 point) What is the output of the following code?

<pre>#include <stdio.h> #define STACK_SIZE 5 int contents[STACK_SIZE]; int top = 0; void push(int v) { contents[top++] = v; } int pop() { return contents[--top]; }</pre>	<pre>int main() { push(1); push(2); push(3); printf("%d ", pop()); push(4); printf("%d ", pop()); printf("%d ", pop()); return 0; }</pre>
---	---

Your answer: _____

- ② (1points) Identify the bug in the following push() function of a stack implementation and briefly explain the issue.

<pre>void push(int value) { contents[top] = value; top++; }</pre>

Your answer:

- ③ (1 points) Why is random access not allowed in stack?

Your answer:

- ④ (2 points) Based on the concept of modularity discussed in class, identify and briefly explain the potential modularity issues in the following code.

```
/* stack.h */
#define STACK_SIZE 100

struct Stack {
    char contents[STACK_SIZE];
    int top;
};

struct Stack *Stack_new(void);
void Stack_free(struct Stack *s);
void Stack_push(struct Stack *s, const char *item);
char *Stack_top(struct Stack *s);
void Stack_pop(struct Stack *s);
int Stack_isEmpty(struct Stack *s);
```

Your answer:

- ⑤ (1 points) Consider the following implementation of the enqueue() function. Identify the bug and provide a corrected version of the code.

```
void enqueue(Queue *q, int v) {  
    if (q->size == QUEUE_SIZE)  
        return;  
    q->rear = (q->rear + 1) % QUEUE_SIZE;  
    q->contents[q->rear] = v;  
    q->size++;  
}
```

Your answer:

Consider the following implementation of the Stack_push() function.

```
void Stack_push(Stack *s, int value) {  
    assert(s != NULL);  
    if (is_full(s))  
        stack_overflow();  
    else  
        s->contents[s->top++] = value;  
}
```

- ⑥ (1 point) If s is NULL, which type of error is it? (Programmer / User)

Your answer: _____

- ⑦ (1 point) If the stack is full, which type of error is it? (Programmer / User)

Your answer: _____

- ⑧ (2 points) Fill in the blank to implement a function that checks if parentheses are balanced using stack.

```
int is_balanced(char *str) {  
    Stack s;  
    make_empty(&s);  
  
    for (int i = 0; str[i]; i++) {  
        if (str[i] == '(')  
            push(&s, '(');  
        else if (blank) {  
            if (is_empty(&s))  
                return 0;  
            pop(&s);  
        }  
    }  
    return is_empty(&s);  
}
```

Your answer: _____

9. Linked Lists

- ① (1 point) What is the time complexity of searching in a linked list?

Your answer: _____

- ② (1 points) Discuss the strengths of a linked list.

Your answer:

- ③ (1 points) Discuss the weaknesses of a linked list.

Your answer:

- ④ (2 points) Fill in the blank to implement a function that reverses a linked list.

```
struct Node* reverse(struct Node *head) {  
    struct Node *prev = NULL, *curr = head;  
    while (curr) {  
        struct Node *next = curr->next;  
        blank = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

Your answer: _____

- ⑤ (2 points) Explain in English (without code) under what condition the following function returns 1.

```

int f(struct Node *head) {
    struct Node *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return 1;
    }
    return 0;
}

```

Your answer:

- ⑥ (1 points) In the Table_free() function below, why does it need nextp?

```

void Table_free(struct Table *t) {
    struct Node *p;
    struct Node *nextp;
    for (p = t->first; p != NULL; p = nextp) {
        nextp = p->next;
        free(p);
    }
    free(t);
}

```

Your answer:

- ⑦ (2 points) Fill in the blank to implement Table_free() function using a while loop.

```
void Table_free(struct Table *t) {  
    struct Node *p = t->first;  
  
    while (p != NULL) {  
        struct Node *nextp = p->next;  
        free(p);  
        blank ;  
    }  
    free(t);  
}
```

Your answer: _____